PoliSense: Discovery & Assessment Pipelines — Technical Design

Nov 8, 2026 · @Darshan Kumar Hegde

Overview

PoliSense has two output pipelines that run sequentially for every Zoho form submission: the **Discovery pipeline** runs immediately on receipt, free of charge and free of LLM calls; the **Assessment pipeline** runs after payment and uses an LLM to produce a detailed eligibility and benefit calculation report. Both pipelines share the same retrieval engine (`pg_retriever.py`) and write their output as a PDF to `Retrievals/` .

	Discovery	Assessment
Trigger	Every new Zoho submission	Post-payment
LLM calls	None	Yes (Gemini Flash)
top_k	20	20
Category filter	Dropped deliberately	Applied
Output	Scorecard PDF — match score + category breakdown	Multi-section PDF — per-policy eligibility + benefit quantum
Entry point	`discovery_pipeline.py`	`assessment_pipeline.py`

The two-tier design keeps the free tier cheap and fast while giving the paid tier the data it needs. The discovery PDF is generated in under a second; the assessment takes several seconds per query due to multiple LLM calls running in parallel.

Shared Retrieval Engine

Both pipelines call `retrieve_policies()` in `pg_retriever.py` . All retrieval decisions are made once here and inherited by both.

Embedding model: BGE `BAAI/bge-small-en-v1.5` (384 dimensions)

BGE requires a task prefix at query time: every query is prefixed with `"Represent this sentence for searching relevant passages: "` before embedding. Documents are stored without a prefix. This asymmetric scheme is a BGE-specific convention — omitting it at query time causes a measurable drop in recall. The model is loaded once per process as a module-level singleton in `postgres_utils.py`; loading it inside `retrieve_policies()` would re-initialise the model on every call.

Vector store: PostgreSQL + pgvector

The `<=>` operator computes cosine distance. The SELECT clause computes `1 - (embedding <=> query_vector)` to return similarity (higher = closer). The `ORDER BY` clause sorts on the raw distance expression so pgvector can use its IVFFlat index; re-expressing as similarity in ORDER BY would prevent index use.

Filter semantics: universal fallback values

Every array filter has a fallback that lets universally-applicable policies always pass:

Filter	Column	Fallback value
`industry`	`applicable_industries TEXT[]`	`'All Sectors'`
`business_size`	`applicable_business_sizes TEXT[]`	`'All'`
`geography`	`applicable_geographies TEXT[]`	`'National'`
`category`	`category TEXT`	None — exact match only

This means a policy tagged `All Sectors` is returned regardless of the industry filter. Without this, nationally-applicable policies would be silently excluded when any filter is active.

Geography uses `ILIKE` with a `%geography%` wildcard rather than exact match. Policy documents often store geographies as `"Karnataka"`, `"Karnataka (Bengaluru)"`, or `"All of Karnataka"` — exact match would miss these variants.

Reranking

After the initial pgvector `ORDER BY` retrieves `top_k` rows, a second scoring pass re-ranks them. Each result's `incentive_details` JSONB is assembled into a text string (incentive name + benefit headline + description + up to three `source_text` snippets) and embedded. The cosine similarity of this incentive-level text against the query vector becomes an `incentive_score`.

Results are then sorted by `(-similarity_score, -incentive_score)`. The purpose: the initial pgvector sort ranks by the main policy embedding, so multiple incentives under the same parent policy appear in an arbitrary order. The rerank sorts siblings by how well each incentive's own text matches the query, so the most relevant incentive surfaces first within each policy group.

Discovery Pipeline

The discovery pipeline (`discovery_pipeline.py`) runs on every new Zoho submission. Its job is to produce a quick-scan PDF showing how well a business profile matches the policy database, without spending money on LLM calls.

Why no LLM calls

The discovery report is a free output. Its value is directional — it shows match strength by category so the client can judge whether the paid report is worth purchasing. Exact eligibility and benefit figures require the LLM; a heuristic score based on cosine similarity is accurate enough for that purpose and runs in well under a second.

Deliberate removal of the category filter

The Zoho form collects an `enquiry_purpose` field (e.g. `"Capital Subsidy"`) which maps to the `category` column. In the assessment pipeline this filter is applied. In discovery it is explicitly removed:

```
teaser_filters = {k: v for k, v in filters.items() if k != "category"}
```

The reason: a business submitting an `enquiry_purpose` of `"Capital Subsidy"` may still be eligible for `"Reimbursement"` , `"Tax Incentive"` , or `"Employment Incentive"` schemes. Applying the category filter in the discovery PDF would artificially narrow the scorecard and could lead a client to believe fewer schemes exist for them than actually do. The discovery PDF is specifically designed to show breadth across all categories.

top_k = 20

The discovery pipeline requests 20 results rather than the default 5 used in the basic retrieval pipeline. This wider net is intentional: the scorecard aggregates results by category and renders per-category match bars, so having more results produces a more accurate category breakdown. With only 5 results, entire categories could be missing from the scorecard.

Scoring logic

The overall score shown in the donut chart is computed entirely in the PDF layer (`discovery_report.py`) from the retrieval results — no additional DB query or LLM call is made:

1. Average cosine similarity of the top-3 results, scaled to 0–100.
2. Up to +15 bonus points if the submitted `industry`, `geography`, and `business_size` each appear in at least one result's matching columns.

The cap at 100 prevents inflated scores. The bonus rewards profile specificity: a client with a detailed profile gets credit for the fact that narrowed results exist for them specifically.

Connection management

The batch function opens one database connection for all queries in the batch and closes it in a `finally` block regardless of per-query failures. Individual query failures are caught and printed without aborting the rest of the batch. This ensures one malformed profile does not block PDF generation for all other submissions in the same polling cycle.

Assessment Pipeline

The assessment pipeline (`assessment_pipeline.py`) runs after payment. It takes the same retrieval results as the discovery pipeline and runs LLM-driven eligibility and benefit analysis on each policy.

Step 1: Cost breakup extraction

Before retrieval, the pipeline calls `extract_cost_breakup(profile)`. The Zoho form has a free-text `project_cost_breakup` field where clients may type something like `"Land: 50L, Civil: 80L, P&M: 200L"`. This text is parsed to extract `land_cost_lakhs`, `civil_cost_lakhs`, and `working_capital_lakhs`, which are needed later for specific benefit calculations (e.g. stamp duty exemptions are calculated on land cost).

This extraction happens before retrieval so that the enriched profile is available throughout the rest of the pipeline without threading it separately.

Step 2: Tier-1 retrieval

Identical to the discovery pipeline except the category filter is applied here. The assessment is a paid, targeted report — the client has stated their `enquiry_purpose`, and including off-category policies would dilute the report's signal.

Step 3: Policy grouping

The retrieval engine returns one row per incentive. A single policy (e.g. `"Karnataka Industrial Policy 2020"`) may have several incentives, each as its own row. Before LLM calls, these rows are grouped by `policy_name` into `PolicyGroup` dicts:

```
{ policy_name, parent_policy_name, max_similarity_score, policy_details,
incentive_rows[] }
```

Grouping is done before LLM calls for two reasons. First, eligibility is a policy-level concept — a company is eligible or not for the policy as a whole, not separately for each incentive. Making one eligibility call per policy group instead of per incentive row reduces LLM calls significantly. Second, grouping makes it possible to aggregate total benefit across all incentives under a policy on the final report.

Step 4: Parallel policy processing

Each policy group is processed independently (eligibility + benefit calculation). These calls are I/O-bound (waiting on the Gemini API), so they are run concurrently using `ThreadPoolExecutor` with `_MAX_WORKERS = 5`:

```
with ThreadPoolExecutor(max_workers=_MAX_WORKERS) as executor:
    futures = {executor.submit(_process_policy_group, group, profile):
group["policy_name"]
               for group in policy_groups}
```

Worker count is set to 5 as a conservative limit. Higher values risk rate-limiting from the Gemini API. Per-future exceptions are caught and logged without aborting the other futures, so a single failed policy group does not block the rest of the report.

Step 5: Report assembly and sort order

Policies are split into three buckets by `overall_eligibility`: `ELIGIBLE`, `PARTIALLY_ELIGIBLE`, `NOT_ELIGIBLE`. Within `ELIGIBLE` and `PARTIALLY_ELIGIBLE`, policies are sorted by `total_potential_benefit_lakhs` descending (highest benefit first). Within `NOT_ELIGIBLE`, policies are sorted by `similarity_score` descending (most relevant first, since no benefit figure exists to sort by).

This sort order is a deliberate presentation decision: the most actionable results appear at the top. A client reading the PDF encounters the highest-value eligible policies first, then the partial-eligibility ones they could potentially close gaps on, then the not-eligible ones for reference.

Eligibility Checker

The eligibility checker (`AssessmentPipeline/eligibility_checker.py`) evaluates one policy's `eligibility_criteria` JSONB against the submitted company profile. It uses a two-phase approach: rule-based checks first, then one LLM call for the remaining free-text criteria.

Why two phases

Some eligibility fields are unambiguously checkable with arithmetic. For example, whether a company's investment meets a minimum threshold is a numeric comparison — there is no interpretation involved. Running an LLM for this would be slower, more expensive, and less reliable than a direct comparison. The LLM is reserved for criteria that require reading natural language, such as location requirements, registration conditions, or exclusion clauses.

Phase 1: Rule-based checks

Four fields are handled deterministically:

Field	Logic
`enterprise_type`	Enum match: `New` , `Existing` , `Expansion` , or `Any`
`minimum_investment`	Numeric comparison against `investment_pm_lakhs` or `total_project_cost_lakhs`
`maximum_investment`	Numeric comparison with same fallback
`maximum_turnover`	Business size band mapped to an upper bound in lakhs; conservative check

For `maximum_turnover` , the profile does not include an exact turnover figure — only a `business_size` band (e.g. `"Small"`). The checker maps each band to a known upper bound (e.g. Small = ₹10 crore). If the band's upper bound is below the threshold, the status is `MET` ; if the band straddles the threshold, it returns `UNCERTAIN` with a note explaining the ambiguity. This avoids falsely clearing a company whose actual turnover could exceed the cap.

Employee count is never in the form, so `minimum_employees` always produces `UNCERTAIN` with a note naming the missing field.

Phase 2: LLM check

The five free-text fields (`location_requirements`, `registration_requirements`, `special_category_benefits`, `exclusions`, `other_requirements`) are bundled into a single Gemini Flash call. Bundling all free-text criteria into one call rather than one call per criterion keeps latency low regardless of how many criteria a policy has.

The prompt includes the verbatim `raw_text` from the policy document alongside the structured criteria fields. The `raw_text` is the authoritative source — the structured fields are summaries extracted by Gemini 2.5 Pro at ingestion time and may have lost detail. If the LLM call fails, all free-text fields fall back to `UNCERTAIN` rather than crashing.

Hard blockers vs. soft blockers

Four fields are defined as hard blockers: `enterprise_type`, `minimum_investment`, `maximum_investment`, `maximum_turnover`. A `NOT_MET` result on any of these produces an `overall_eligibility` of `NOT_ELIGIBLE`. A `NOT_MET` on any other field (from the LLM phase) produces `PARTIALLY_ELIGIBLE` instead.

This distinction matters for the report structure: `NOT_ELIGIBLE` policies go into Section C with only a gap summary and no benefit calculation. `PARTIALLY_ELIGIBLE` policies go into Section B with full benefit calculations, because there is still a path to eligibility.

Benefit Calculator

The benefit calculator (`AssessmentPipeline/benefit_calculator.py`) processes each incentive row and returns a calculated benefit quantum in INR lakhs. Like the eligibility checker, it uses a rule-based-first, LLM-fallback approach.

Cost breakup extraction

Before the retrieval step, `extract_cost_breakup()` parses the `project_cost_breakup` free text from the profile. The extractor runs a regex pass first:

1. Split on common delimiters (`,`, newline, `;`, `|`).
2. Match each segment against keyword patterns (`land`, `civil/building/construction`, `working capital/margin money`).
3. Parse the first monetary amount from each matching segment using a regex that handles crore/lakh/INR unit variants.

If any of the three components (`land_cost_lakhs` , `civil_cost_lakhs` , `working_capital_lakhs`) is not found by regex, a Gemini Flash call is made with the raw text and the top-level investment figures as cross-references. The LLM fallback is scoped to only the missing components; found components are not re-extracted. This minimises LLM usage — most well-formatted inputs are handled entirely by regex.

Rule-based slab calculation

For each `QuantifiedBenefit` slab in an incentive's `quantified_benefits` list, the calculator first attempts a rule-based calculation:

Benefit type	Rule
`fixed_grant` , `fee_waiver` , `loan`	Use `value_numeric` directly if present
`percentage_subsidy`	`value_numeric % × investment_pm_lakhs` (or total project cost)
`reimbursement` (percentage)	Same as percentage_subsidy
`employment_incentive`	Always `UNCERTAIN` — employee count not in form
`interest_reduction`	Always `UNCERTAIN` — loan amount not in form

After arithmetic, caps are applied: if `cap_numeric` is present in the slab, `effective_amount_lakhs = min(calculated, cap)` . Both values are stored so the report can show the uncapped calculation and the applied cap separately.

Slabs that cannot be rule-calculated (no numeric value, unhandled type, or missing required profile field) are queued for the LLM.

LLM-based calculation

All pending slabs from a single incentive are sent in one Gemini Flash call, not one call per slab. The prompt provides the verbatim `source_text` from the policy document for each slab as the authoritative figure. The LLM is instructed to show step-by-step arithmetic and express all amounts in lakhs. The response is validated against a Pydantic schema; if parsing fails, all pending slabs fall back to `UNCERTAIN` .

Confidence levels

Every slab result carries a `confidence` field (`HIGH` , `MEDIUM` , `LOW`). Rule-based results with all inputs present are `HIGH` . Rule-based results using an estimated or fallback input are `MEDIUM` . LLM results where key inputs are missing, and all fallback results, are `LOW` .

The PDF renders confidence in colour: green for HIGH, amber for MEDIUM, red for LOW, so readers can judge which figures to rely on.

LLM Usage

Model choices

Two Gemini models are used at different stages:

Stage	Model	Reason
Ingestion (PDF extraction)	Gemini 2.5 Pro	High-quality structured extraction from raw PDFs; runs once per document, cost is acceptable
Assessment (eligibility + benefit calc)	Gemini 2.5 Flash	Runs multiple times per query at report generation time; speed and cost matter more than marginal accuracy gain

The Flash model is initialised via `make_gemini_flash()` in `AssessmentPipeline/prompts.py`. Pro is initialised in `IngestionPipeline/llm_extractor.py`. They are kept in separate modules so neither pipeline imports the other's LLM dependency.

Temperature = 0

All LLM calls across both pipelines use `temperature=0`. The rationale: these are deterministic analysis tasks — eligibility is either met or not, and a benefit calculation should produce the same number given the same inputs. Temperature > 0 would introduce variance that makes the results non-reproducible and harder to audit. It also prevents the model from hedging with invented qualifications not present in the source text.

JSON schema mode (structured output)

All LLM calls use `with_structured_output(schema=..., method="json_schema")`. This forces the model to return a JSON object conforming to a Pydantic-derived schema rather than free text. The benefits:

1. The calling code can parse the response directly into a typed Pydantic object without regex or manual parsing.

2. The schema constrains the model's output vocabulary — for example, `status` can only be `MET`, `NOT_MET`, or `UNCERTAIN`. Without this, the model might return `"PARTIALLY MET"` or `"N/A"`, which would require normalisation.
3. If the model cannot produce valid JSON, the call raises an exception cleanly, triggering the fallback path rather than silently returning malformed data.

Source text anchoring

Both the eligibility and benefit prompts include the verbatim text from the original policy document alongside the structured fields. For eligibility, this is `eligibility_criteria.raw_text` — the exact sentences from the PDF's eligibility section, stored at ingestion time. For benefit calculation, each `QuantifiedBenefit` slab carries a `source_text` field with the verbatim sentence(s) that state the benefit amount.

The prompt instructs the model to use the source text as the authoritative figure and not invent amounts not stated there. This is a deliberate constraint: the structured fields (e.g. `value_numeric`) are summaries that could have lost precision or context during ingestion-time extraction. Anchoring to verbatim text gives the model a way to catch extraction errors and reason from the original language.

Output Layer — PDF Generation

Both pipelines produce PDFs using ReportLab's `platypus` layout engine (`PDF_utils/discovery_report.py` and `assessment_report.py`). ReportLab was chosen over a template-to-HTML-to-PDF approach because it gives precise control over layout without a headless browser dependency, and the reports need to work in a server environment without a display.

Discovery PDF

The discovery PDF is a single-page scorecard with five sections:

1. Header bar with business name, report date, and a unique report ID.
2. Intro paragraph naming the business and what was scored.
3. Score section: a donut chart showing the overall score (0–100) and a per-category breakdown table with progress bars and High/Medium/Low labels.
4. Gap section: a locked amber callout showing how many eligibility conditions were found in the results, with placeholder locked bars to preview gap content.
5. CTA section: a dark panel with the payment link and price to unlock the full report.

The discovery PDF deliberately does not name which schemes were found or show any benefit figures. The gap bars are labelled `LOCKED` and use a placeholder width (55%) rather than real values. This is intentional — the specific schemes and amounts are reserved for the paid report.

Assessment PDF

The assessment PDF is a multi-page structured report with a cover page followed by three sections:

- **Cover page**: client profile table, summary stats (eligible count, partial count, not-eligible count, total potential benefit), and a global missing-fields callout if any profile fields are absent.
- **Section A — Eligible**: one entry per eligible policy with an eligibility criteria table (MET/NOT_MET/UNCERTAIN rows colour-coded green/red/amber), a per-incentive benefit slab table showing calculated amounts and calculation steps, and a policy-level total.
- **Section B — Partially Eligible**: same structure as Section A, plus a missing-fields callout on each policy naming the fields that would resolve UNCERTAIN checks.
- **Section C — Not Eligible**: a compact table listing all not-eligible policies with their primary gap and what would need to change. No benefit detail is shown here.

Within Sections A and B, policies are ordered by `total_potential_benefit_lakhs` descending — the same sort used in the report assembly step — so the highest-value opportunities appear first.

Font handling

ReportLab's built-in PDF fonts (Helvetica, Times, Courier) do not include the ₹ symbol. The discovery and assessment reports both attempt to register Arial from `C:\Windows\Fonts\arial.ttf` at module load time and fall back to Helvetica if the file is not present. Any field that may display a rupee symbol uses the registered Unicode font rather than Helvetica.

Data Model

The assessment pipeline uses a set of Pydantic models (`AssessmentPipeline/models.py`) that are distinct from the ingestion-time extraction models in `IngestionPipeline/llm_extractor.py`. The separation keeps the two

concerns independent: ingestion models represent what was extracted from PDFs; assessment models represent the results of evaluating a business against those extractions.

Flow through the pipeline

Stage	Input	Output type
Retrieval	Query + profile	`list[dict]` (raw DB rows)
Grouping	`list[dict]`	`list[PolicyGroup dict]`
Eligibility check	`policy_details` JSONB + profile	`PolicyEligibilityResult`
Benefit calc (per incentive)	`incentive_details` JSONB + profile	`IncentiveResult`
Per-policy assembly	`PolicyEligibilityResult` + `list[IncentiveResult]`	`FullPolicyResult`
Report assembly	`list[FullPolicyResult]` + profile + query	`Tier2Report`
PDF generation	`Tier2Report`	PDF file

Key types

`CriterionCheck` — one evaluated eligibility criterion. Holds the criterion name, what the policy requires, what the company has, a status (`MET` / `NOT_MET` / `UNCERTAIN`), an exact gap description for `NOT_MET` , and an actionable recommendation.

`PolicyEligibilityResult` — wraps a list of `CriterionCheck` objects and derives `overall_eligibility` . Also carries `missing_profile_fields` : the set of profile fields whose absence caused `UNCERTAIN` statuses.

`BenefitSlabResult` — one calculated benefit slab. Holds the verbatim `source_text` from the document, `calculated_amount_lakhs` before the cap, `effective_amount_lakhs` after, a `cap_applied` flag, `calculation_steps` as a readable string, and `confidence` .

`IncentiveResult` — wraps all slabs for one incentive and derives overall `applicability` . Holds the sum of `effective_amount_lakhs` across applicable slabs as `total_potential_benefit_lakhs` .

`FullPolicyResult` — combines one `PolicyEligibilityResult` with all `IncentiveResult` objects for the policy. Separates incentives into `applicable_incentives` and `not_applicable_incentives`. Aggregates `total_potential_benefit_lakhs` and the union of `missing_profile_fields` across eligibility and all incentives.

`Tier2Report` — the top-level output. Contains three sorted lists of `FullPolicyResult` (eligible, partially eligible, not eligible), a global union of missing fields, and an aggregated total potential benefit across eligible and partially eligible policies.